Team name: Ville Karppinen

Project Report

Smart Wildlife Monitoring System

IoT Project

Metropolia University of Applied Sciences

27.8.2026

Abstract

This project focused on developing an embedded motion-triggered camera system using the OpenMV N6 development board. The system combined a PAG7936 RGB camera and a FLIR Lepton 3.5 thermal camera to detect movement and record events. A central requirement of the project was to maintain a circular pre-event image buffer in RAM so that recorded video would contain footage from several seconds before motion was detected.

The system was implemented in MicroPython using the OpenMV camera interfaces. A five-second circular RAM buffer operating at five frames per second was developed for pre-event recording. Thermal frame differencing was used for motion detection, while the RGB camera provided higher-resolution event video. Both RGB and thermal recordings were stored as MJPEG files on a microSD card. Additional functionality was developed for MJPEG timing and AVI index correction, persistent logging, storage quotas, memory management, camera calibration handling, and hardware watchdog recovery.

During development, several hardware, firmware, memory-management, camera-interface, and video-container problems were investigated. Long-duration stress testing demonstrated stable operation under both isolated and full-system conditions. An approximately eleven-hour test of the camera and memory-management pipeline completed without crashes or memory-related failures while network uploading was disabled. More importantly, a separate full-system stress test ran successfully for more than ten hours with all major features enabled, including camera monitoring, dual-camera recording, RAM buffering, watchdog operation, and network file uploads. When the Wi-Fi hotspot connection was later lost, the system eventually became unresponsive, the hardware watchdog automatically rebooted the device, and normal operation resumed without manual intervention.

The resulting implementation demonstrates that an OpenMV N6 can be used to implement a compact embedded monitoring system combining thermal motion detection, RAM-based pre-event buffering, RGB and thermal video recording, persistent local storage, network file transfer, and fault-recovery mechanisms. The system was also successfully integrated with an AWS backend for S3 storage and automated wildlife recognition using Amazon Rekognition.

Contents

1 Introduction

Embedded camera systems are commonly required to react to events rather than continuously record video. One limitation of purely event-triggered recording is that recording normally begins only after an event has already been detected. This can result in the beginning of an event being lost.

The objective of this project was to develop a motion-triggered camera system around the OpenMV N6 development board. An important project requirement was to maintain recent camera frames in RAM using a circular buffer. When motion is detected, these buffered frames can be written to the beginning of the recording, allowing the resulting video to show what occurred before the trigger event.

The project developed beyond basic motion detection into a dual-camera system using a PAG7936 RGB camera and a FLIR Lepton 3.5 thermal camera. Thermal imaging was used for motion detection, while the RGB camera was used to provide higher-resolution visual information. Both cameras were also incorporated into the event-recording process.

The implementation required solving several embedded-system challenges. These included limited available memory, camera-interface behavior, simultaneous camera operation, MJPEG file compatibility, long-term memory stability, thermal-camera calibration events, storage management, asynchronous operation, and recovery from software failures.

Development was carried out iteratively. Individual features were first tested separately and were then integrated into the complete system. A development log was maintained throughout the project to document experiments, problems, solutions, architectural changes, and test results.

1.1 Use of AI Tools

ChatGPT was used as a supporting tool throughout the project for brainstorming, code review, technical-language proofreading, and report-structure planning. Generated suggestions were reviewed against the implemented code, measured test results, and project documentation, and the author retains full responsibility for the final technical content.

2 Hardware and Development Environment

2.1 OpenMV N6

The OpenMV N6 was selected as the main embedded development platform. Development was performed primarily in MicroPython using OpenMV IDE and the OpenMV camera APIs.

Initial testing concentrated on learning the OpenMV development workflow, verifying camera operation, investigating internal flash and microSD storage, and testing example applications. Early tests also revealed that excessive serial output could negatively affect OpenMV IDE responsiveness. Continuous FPS printing eventually caused the IDE to freeze, while continuous image capture without excessive serial output remained stable. This resulted in debug output being kept relatively limited during later development.

The firmware was eventually migrated from OpenMV firmware 4.8.1 to version 5.0.0. The migration required changes to the camera initialization code because of differences in camera API and CSI behavior.

2.2 PAG7936 RGB Camera

The PAG7936 was used as the primary RGB imaging sensor.

The camera supports several resolutions, including 320 x 200, 640 x 400, and 1280 x 800. The final implementation uses 640 x 400 RGB565 images for the circular RAM prebuffer and switches the camera to 1280 x 800 when live event recording begins.

Automatic gain, exposure, and white balance are enabled to improve image usability under changing conditions.

Memory testing showed that the selected buffer resolution was an important design decision. A five-second RGB565 prebuffer at five frames per second contains 25 frames. At 640 × 400 resolution, the prebuffer requires approximately 12.21 MiB of SDRAM, while the same prebuffer at 1280 × 800 would require approximately 48.83 MiB. The higher-resolution buffer exceeded the available MicroPython heap because memory was also required by the runtime, camera frame buffers, and other application data. Therefore, 640 × 400 was selected as the practical prebuffer resolution.

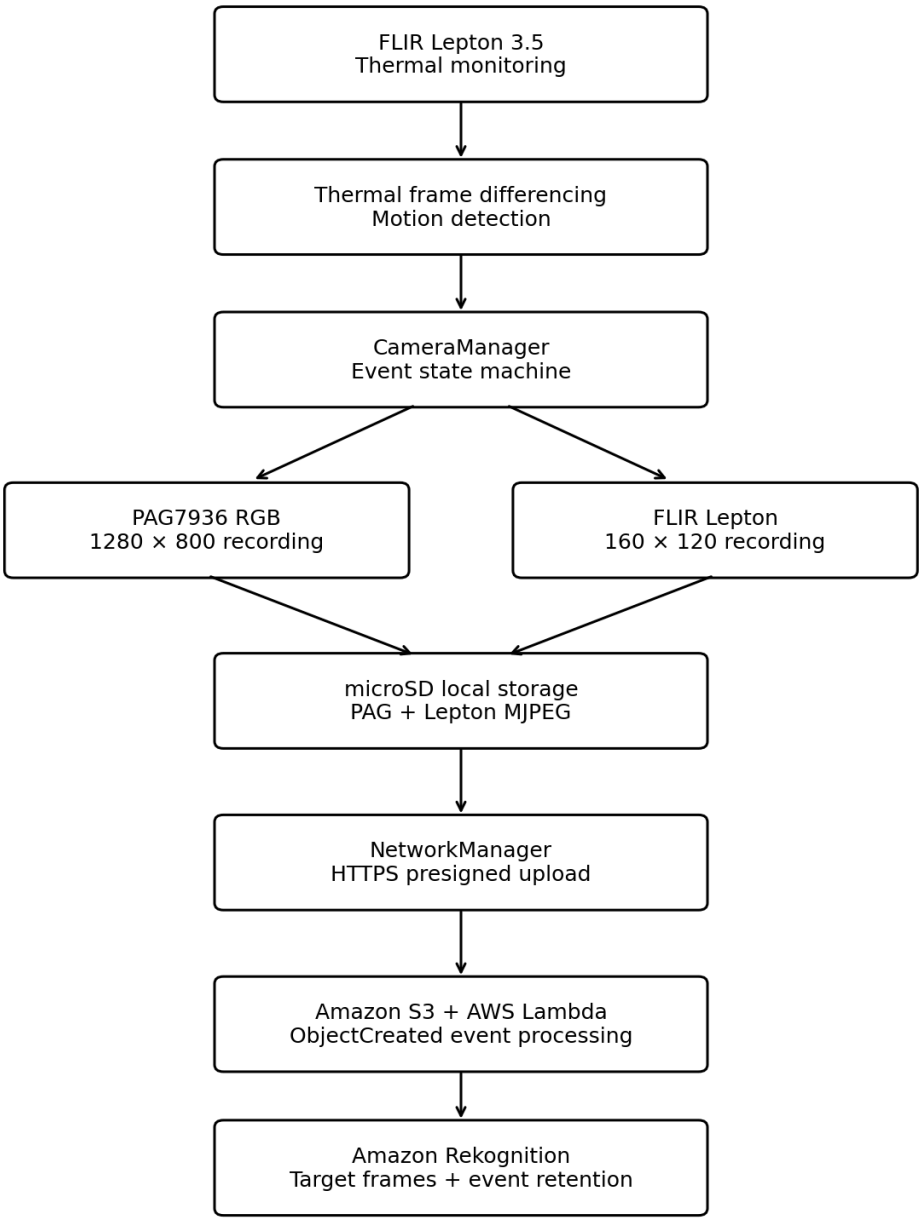
2.3 FLIR Lepton 3.5 Thermal Camera

The second camera used in the project was a FLIR Lepton thermal camera. The Lepton operates at a native resolution of 160 x 120 pixels.

The camera was configured using the OpenMV CSI interface and grayscale output. Radiometric operation was enabled and the thermal image was mapped to a configurable temperature range. The current configuration represents temperatures between 20–40 °C in the grayscale image.

The thermal camera became the primary sensor used for motion detection. This allows the system to react to thermal changes associated with people or other warm moving objects instead of relying only on visible-light image changes.

3 Software Architecture



1. Figure: System architecture and embedded-to-cloud processing flow.

The thermal camera provides the event trigger, while both cameras create local recordings. The files are uploaded through NetworkManager, and AWS processes completed S3 objects without placing permanent cloud credentials on the embedded device.

3.1 Modular Structure

The software was gradually refactored from an initially monolithic camera implementation into separate modules with clearly defined responsibilities.

- CameraBase contains functionality shared by both cameras, including circular-buffer operations and MJPEG file creation.
- CameraPag contains PAG7936 initialization, RGB buffering, image scaling, recording, and resolution switching.
- CameraLepton contains Lepton initialization, thermal buffering, motion detection, recording, and Flat-Field Correction handling.
- CameraManager coordinates the two cameras and manages the recording state machine.
- FileManager handles directories, file numbering, storage calculations, and MJPEG file corrections.
- NetworkManager handles Wi-Fi connectivity, network recovery, time synchronization, and asynchronous transfer of recorded MJPEG files.
- LogManager provides persistent timestamped logging.
- TimeManager provides local timestamp handling.
- Watchdog manages the hardware watchdog and reset-cause logging.
- Configuration classes separate adjustable parameters from operational code.

This modular architecture separates camera control, networking, file management, logging, time management, watchdog operation, and configuration into clearly defined components, making the implementation easier to maintain and extend.

3.2 Concurrent Runtime Tasks

The application uses MicroPython asyncio to coordinate the main runtime operations [14].

At startup, the file, logging, network, watchdog, and camera components are initialized. The application then runs several tasks concurrently. These include watchdog servicing, camera frame-buffer updates, motion monitoring, and networking operations.

The RGB and thermal frame buffers are themselves updated concurrently. This design allows the application to perform periodic background operations without relying on one large sequential main loop.

Video recording is intentionally handled differently. The recording state machine is synchronous because recording is a time-critical operation that requires coordinated control of the camera system. The hardware watchdog is therefore explicitly serviced during the blocking recording loop.

3.3 AWS Backend Software Architecture

The embedded application is complemented by a modular AWS backend located in the openmv-wildlife-backend directory. The backend separates request routing, upload URL generation, validation, S3 event handling, storage operations, image recognition, configuration, and recognition targets into independent modules.

- lambda_function.py is the main AWS Lambda entry point. It distinguishes API Gateway upload requests from S3 ObjectCreated events and routes them to the correct handler.
- upload_api.py generates temporary presigned S3 PUT URLs, allowing the OpenMV device to upload PAG and Lepton MJPEG files without storing permanent AWS credentials.
- validation.py validates the camera ID, event ID, and sensor type before an upload URL is generated.
- s3_event_handler.py handles completed S3 uploads, identifies the sensor and event, and coordinates PAG and Lepton processing.

- s3_storage.py handles S3 operations, including checking object existence, loading recordings, saving detected frames, and deleting rejected events.
- image_recognition.py extracts JPEG frames from PAG recordings and uses Amazon Rekognition to detect configured targets.
- config.py contains the AWS backend configuration, including the S3 bucket, upload prefix, and presigned URL expiration time.
- recognition_targets.json contains the configurable target labels used by the image-recognition logic.

4 Circular RAM Prebuffer

4.1 Purpose

A major requirement of the project was to capture video from before the actual motion trigger.

A conventional event-triggered camera starts recording only after movement has been detected. The circular prebuffer solves this problem by continuously retaining the most recent camera frames in RAM.

The pre-event buffers operate according to the First-In, First-Out (FIFO) principle [11]. When a buffer reaches its maximum capacity of 25 frames, adding a new frame causes the oldest frame to be removed first. This allows the circular buffer to continuously retain only the most recent five seconds of camera data while keeping memory usage bounded.

The current buffer configuration stores five seconds of video at five frames per second. Each camera therefore maintains up to 25 pre-event frames.

4.2 Circular Buffer Implementation

The buffer is implemented as a fixed-size Python list. New frames are inserted using an index that wraps back to the beginning of the list after the final position has been reached.

Once the buffer has been filled, every new frame overwrites the oldest stored frame. Memory consumption therefore remains bounded instead of continuously increasing.

When recording begins, the frames are retrieved from the buffer in chronological order. Unused buffer positions are ignored if an event occurs before the buffer has been completely filled.

Testing confirmed that the PAG7936 five-second prebuffer contained all 25 expected frames when an event was triggered.

4.3 RGB Prebuffer Scaling

The RGB prebuffer uses 640 × 400 frames because storing five seconds of 1280 × 800 RGB565 images would require approximately 48.83 MiB, compared with approximately 12.21 MiB at 640 × 400. The higher-resolution prebuffer would exceed the available MicroPython heap.

However, live RGB recording uses 1280 x 800. To combine both sections into one recording, the buffered 640 x 400 images are scaled to 1280 x 800 before being written to the MJPEG file.

A reusable 1280 x 800 RGB565 image is allocated during camera initialization. Reusing this image avoids repeatedly allocating a large temporary image while writing buffered frames and helps reduce heap fragmentation.

4.4 Prebuffer Catch-Up

Writing the complete prebuffer to storage takes time. Without compensation, this could introduce a gap between the end of the buffered section and the beginning of live recording.

To reduce this gap, new camera frames are captured periodically while the buffered frames are being written. These catch-up frames are temporarily stored and appended after the original prebuffer. Live recording can then continue from a more recent point in time.

5 Thermal Motion Detection

5.1 Frame Differencing

Motion detection is based on thermal frame differencing using the image-processing functionality provided by OpenMV [1], following the general frame-differencing approach demonstrated in the official OpenMV examples [4].

A background thermal frame is maintained in memory. The current thermal frame is compared with this reference using image differencing. A histogram is then calculated from the resulting difference image.

Instead of relying on a single pixel or simple average, the implementation compares selected histogram percentiles. The current configuration uses the 90th and 99th percentiles. The difference between these values is compared with a configurable trigger threshold.

This method allows the system to identify significant changes in the thermal image while reducing the influence of smaller variations.

5.2 Adaptive Background

The thermal background reference is periodically updated so that slow environmental changes do not continuously trigger motion detection.

After a configurable number of frames, the current frame and background reference are blended and the result becomes the updated background. This creates an adaptive reference rather than assuming that the thermal environment remains constant indefinitely.

5.3 Flat-Field Correction Handling

The FLIR Lepton was integrated using the OpenMV thermal-camera interface and official Lepton examples [1], [5]. The camera periodically performs Flat-Field Correction (FFC) for self-calibration. During FFC, the internal shutter temporarily interrupts normal thermal imaging [10].

During testing, FFC was found to interfere with frame-difference motion detection. The calibration temporarily changes the thermal image significantly enough to resemble movement to the frame-difference algorithm.

FFC status monitoring was therefore added to the implementation. Thermal motion detection is suspended while calibration is active. After calibration finishes, the system waits for the thermal image to stabilize before motion detection is enabled again.

Initial testing used a 2–4 second recovery period, but this was insufficient and caused false triggers immediately after calibration. Increasing the recovery period to five seconds eliminated these immediate false triggers during testing.

After recovery, the old thermal background is replaced with a new reference captured from the stabilized image. This prevents post-calibration frames from being compared with a background captured before calibration.

Initial testing used a 2-4 second recovery period, but this was insufficient and caused false triggers immediately after calibration. Increasing the recovery period to five seconds eliminated these immediate false triggers during testing.

After recovery, the old thermal background is replaced with a new reference captured from the stabilized image. This prevents post-calibration frames from being compared with a background captured before calibration.

6 Event Recording

6.1 Recording State Machine

The recording workflow is controlled by a state machine in CameraManager.

The PREPARE state creates the RGB and thermal MJPEG files and reserves the next video number. The PREBUFFER state writes the buffered RGB and thermal frames to their respective recordings. The RECORDING state performs live recording and periodically checks whether thermal motion is still present. Finally, the FINALIZING state closes and corrects the video files, records diagnostic information, clears the circular buffers, and restores the normal camera state.

The state machine is reset to PREPARE even if an exception occurs, preventing an interrupted recording from leaving the software permanently in an intermediate recording state.

6.2 Recording Duration

Live recording continues while motion remains active.

The system periodically checks thermal motion during recording. Every time movement is detected, the last-motion timestamp is updated. If no further motion is detected for five seconds, recording ends.

A maximum recording duration of one minute is also configured. This prevents a continuous trigger condition from creating an indefinitely growing recording.

6.3 Dual-Camera Recording

Separate MJPEG files are created for the PAG7936 and Lepton cameras, but both belong to the same event.

The RGB recording provides higher-resolution visual information, while the thermal recording preserves the thermal activity associated with the event.

The PAG7936 was measured at approximately 4.84 FPS during live recording against a target of 5 FPS, demonstrating that the selected recording rate could be maintained closely in the tested implementation.

7 MJPEG File Handling

7.1 Local Recording

Recorded videos are stored on the microSD card in the /sdcard/motion_capture directory.

Sequential file numbering is used to avoid overwriting previous recordings. During startup, existing files are inspected and the next available number is selected.

7.2 Timing Correction

During development, MJPEG files could contain incorrect playback timing information. This caused media players to report incorrect durations and frame rates. OpenMV's MJPEG implementation stores timing and frame-count information at fixed positions in the generated AVI/MJPEG header and updates these fields during synchronization [15].

A custom timing correction was implemented in FileManager using the same header-field locations used by the OpenMV MJPEG implementation. After recording, the actual number of frames and recording duration are used to recalculate and update the relevant AVI timing fields.

Testing later identified an incorrect AVI stream-length calculation. After changing the stream length to use the actual number of recorded frames, the reported frame count, five FPS playback rate, and video duration matched correctly.

7.3 AVI Index

Another issue was that OpenMV-generated MJPEG files did not contain the AVI idx1 index expected by media players.

A custom index-generation function was implemented. The function scans the MJPEG frame chunks, records their positions and sizes, appends an idx1 chunk, and updates the RIFF file size.

After this correction, media players could identify the duration immediately and seek through recordings without first rebuilding the missing AVI index.

7.4 Network Management and File Upload

The NetworkManager module separates network connectivity and file-transfer functionality from the camera subsystem. Wi-Fi connectivity was implemented using the OpenMV networking interfaces and official Wi-Fi examples [1], [6].

It initializes the OpenMV N6 Wi-Fi station interface, connects to the configured wireless network, and synchronizes the real-time clock using NTP. After NTP provides UTC time, the existing `TimeManager` converts the clock to Finnish local time so that file and log timestamps use the correct local time.

Network recovery is handled automatically. If the Wi-Fi connection is unavailable, `NetworkManager` attempts to reconnect using configurable retry delays and timeouts. If normal reconnection fails, the Wi-Fi radio interface is

power-cycled and another reconnection attempt is made. Successful reconnection also triggers a new time synchronization.

The module also runs an asynchronous upload task. After a startup delay that allows the camera interfaces and prebuffer to stabilize, it periodically checks for recorded MJPEG files. Pending files are uploaded individually. A file is deleted from the microSD card only after the upload has succeeded; failed uploads therefore remain available for a later retry. Post-upload cleanup is used to release memory and give the TLS/network stack time to free resources.

For each recording, the file manager provides event and sensor metadata. `NetworkManager` combines this information with the configured camera identifier and requests a temporary presigned HTTPS upload URL. The MJPEG file is then streamed directly from the microSD card over an asynchronous TLS connection in configurable blocks instead of loading the complete recording into RAM. A timeout is applied to network drain operations so that a stalled transfer can be aborted rather than blocking indefinitely.

The upload is considered successful only when the remote service returns HTTP status 200. Upload duration and average transfer speed are calculated for diagnostics. Exceptions are handled so that an unsuccessful transfer returns control to the application without deleting the original recording. This networking design is important for long-running embedded operation because camera processing, local storage, and file transfers must coexist within the limited memory and processing resources of the OpenMV N6.

8 AWS Backend and Cloud Processing

8.1 Upload API and Presigned URLs

The embedded device does not store permanent AWS credentials. Instead, temporary presigned Amazon S3 PUT URLs are generated for individual uploads [7].

For each PAG7936 or Lepton recording, NetworkManager sends the camera identifier, event identifier, and sensor type to an API Gateway endpoint. AWS Lambda validates the request and generates a temporary presigned Amazon S3 PUT URL for exactly one MJPEG object. The OpenMV N6 then uploads the recording directly to Amazon S3 over HTTPS.

PAG and Lepton recordings belonging to the same event use a shared S3 event prefix. The object key contains the camera identifier, a zero-padded event number, and the sensor filename, for example uploads/camera_001/event_00042/pag.mjpeg and uploads/camera_001/event_00042/lepton.mjpeg.

8.2 S3 Event Processing

A successful S3 upload generates an ObjectCreated event that invokes the Lambda backend [8].

The Lambda entry point distinguishes API Gateway requests from S3 events and routes them to separate processing functions. The uploaded object key is parsed to recover the camera identifier, event number, and sensor type.

PAG and Lepton files are handled differently. PAG recordings are downloaded by Lambda and their JPEG frames are extracted from the MJPEG stream. Selected frames are analyzed with Amazon Rekognition. Lepton recordings are not sent to Rekognition; instead, the backend checks whether the matching PAG object exists for the same event.

8.3 Wildlife Recognition and Event Retention

Amazon Rekognition analyzes selected PAG frames using label detection [9]. The backend compares the returned labels and confidence values with the configured wildlife targets.

Frames containing configured targets are saved as individual JPEG images under target-specific S3 paths such as uploads/animals/dog/camera_001/event_00005/frame_00000.jpg.

The event directory acts as the synchronization point between the two sensor uploads. If PAG analysis finds a configured target, the event is retained. If no configured target is found, the complete event prefix is deleted, which also removes a Lepton file if it has already arrived. When a Lepton upload is processed, it is retained only when the matching PAG recording exists; otherwise the rejected event is deleted.

8.4 Current Backend Status

End-to-end testing has confirmed successful presigned URL requests, direct MJPEG uploads from the OpenMV N6 to Amazon S3, Lambda invocation from S3 ObjectCreated events, PAG frame extraction, Amazon Rekognition label detection, and storage of detected target frames. CloudWatch logging is used to distinguish upload-URL requests from S3 processing events and to record frame-analysis results. Additional testing is still required for failure cases and closely timed PAG and Lepton uploads.

9 Storage and Logging

9.1 Storage Allocation

The microSD card is logically divided between recordings and logs.
Before a new recording begins, the application checks whether the reserved video storage area has sufficient capacity. If the video quota has been reached, the recording is not started.

Based on measured 60-second recordings, one complete dual-camera event required approximately 75 MB: approximately 74 MB for the PAG7936 video and 1.2 MB for the Lepton video. On approximately 29.6 GB of usable storage,

this was estimated to provide capacity for roughly 410 complete one-minute dual-camera events.

- 85% for video recordings
- 10% for log files
- 5% as an unused safety margin

9.2 Persistent Logging

LogManager provides persistent logging on the microSD card.

Log entries contain a timestamp, severity level, and message. The implementation provides INFO, WARNING, and ERROR levels.

Individual log files are limited to 256 KiB. When the active file reaches this size, a new sequential log file is opened.

The logger also respects the reserved log-storage quota. When additional log data would exceed this quota, the oldest inactive log is automatically removed. The log file currently being written is protected from deletion.

9.3 Local Time

Network time synchronization initially provides UTC time. TimeManager converts this to Finnish local time and handles the winter- and summer-time offsets.

This allows logs and filesystem timestamps to correspond to local Finnish time instead of remaining in UTC.

10 Reliability and Memory Management

10.1 Memory Constraints

Memory management was one of the most important technical challenges in the project.

The circular image buffers, large RGB frames, camera drivers, MJPEG encoding, and other runtime operations all compete for limited embedded memory.

Early ImageIO experiments demonstrated that repeated buffering operations could exhaust the fast framebuffer memory. A frame-based circular buffer was therefore selected instead.

Resolution testing also demonstrated why memory usage had to be considered during system design. A five-second 1280 x 800 RGB565 prebuffer was too large, whereas the 640 x 400 implementation was reliable.

10.2 Garbage Collection

Several approaches to garbage collection were tested during development.

Manual collection was initially used at memory-intensive points. However, interactions between garbage collection, camera buffering, and other operations required further investigation.

The later implementation configured MicroPython's automatic garbage-collection allocation threshold so that collection could occur proactively [12]. Manual garbage-collection calls were reduced where possible.

An approximately eleven-hour overnight stress test of the camera and memory-management pipeline completed without crashes or memory-related failures. During this test, camera processing, circular buffering, FFC handling, and video recording continued to operate successfully.

10.3 Hardware Watchdog

A hardware watchdog was added to recover the system if the application becomes unresponsive [13].

The watchdog currently uses a 30-second timeout and is normally fed every five seconds. Because video recording contains a synchronous processing loop, the watchdog is also explicitly fed during recording.

The system records the reset cause during startup. This allows a watchdog-triggered reboot to be distinguished from power-on, software, hardware, or other reset types.

This mechanism proved useful during extended testing because the device could automatically reboot and resume operation after a failure.

11 Testing and Development Challenges

11.1 microSD Card Problems

Early development was interrupted by repeated failures when writing recordings to the microSD card. The /sdcard mount was missing and attempts to access it produced an ENODEV error.

Testing with a second card confirmed that the application and camera implementation were not responsible. The original card was subsequently reformatted using a PC and became usable again.
This demonstrated the importance of isolating storage hardware problems from software problems before changing application code.

11.2 USB Connectivity

Another hardware-related problem occurred when the OpenMV N6 stopped appearing in OpenMV IDE and Windows Device Manager.

The USB cable was verified using another device and was found to support data transfer. Testing then isolated the problem to the USB hub. Connecting the OpenMV N6 directly to the laptop restored reliable communication.

11.3 Firmware and Camera Interface

Firmware behavior also had a significant effect on development.
During the transition to OpenMV firmware 5.0.0, camera initialization and CSI behavior required investigation. At one stage, initializing the Lepton prevented subsequent PAG7936 captures and caused frame-capture timeouts.

Several camera architectures were tested while investigating these limitations. The camera subsystem was repeatedly refactored as more was learned about the behavior of the OpenMV firmware and the two sensors.

The current implementation initializes separate CSI camera objects and coordinates the two cameras through CameraManager.

11.4 Thermal Calibration

Periodic Lepton FFC calibration initially produced false motion events. Detecting the FFC state, suspending motion detection, adding a five-second recovery period, and rebuilding the background reference solved the immediate post-calibration false-trigger problem observed during testing.

11.5 Long-Term Stability

Long-duration testing was used throughout development instead of relying only on short functional tests.

An approximately eleven-hour stress test was first completed with the network upload task disabled. During this test, camera processing, circular buffering, thermal motion detection, FFC handling, video recording, and memory management operated continuously without crashes or memory-related failures.

A separate full-system stress test was later completed with all major features enabled. The system operated continuously for more than ten hours while performing motion monitoring, dual-camera recording, RAM buffering, file management, watchdog operation, and network file uploads.

During the test, video recording and file uploads operated concurrently without interrupting each other. Near the end of the test, the mobile phone providing the Wi-Fi hotspot was moved to another room. After the wireless connection was lost, the embedded system eventually became unresponsive. The hardware watchdog detected the failure and automatically rebooted the device.

After rebooting, the application initialized automatically, recovered its normal operating state, and continued operation without manual intervention.

The test demonstrated that the complete embedded system can operate continuously for more than ten hours with all major features enabled. It also demonstrated that the hardware watchdog provides effective automatic recovery when a runtime failure occurs. Network-loss handling itself remains an area for further improvement and testing.

12 Results

The project successfully produced a functional embedded motion-triggered camera system on the OpenMV N6.

The five-second PAG7936 circular prebuffer was verified to contain all 25 expected frames. The selected 640 × 400 RGB565 prebuffer requires approximately 12.21 MiB for 25 frames, compared with approximately 48.83 MiB at 1280 × 800. Live PAG7936 recording achieved approximately 4.84 FPS against the configured five FPS target.

Long-duration tests provided additional evidence of system stability. The camera and memory-management pipeline completed an approximately

eleven-hour stress test without crashes when network uploading was disabled. A separate full-system stress test demonstrated more than ten hours of continuous operation with camera monitoring, dual-camera recording, RAM buffering, watchdog operation, and network file uploads enabled. After loss of the Wi-Fi hotspot connection caused the system to become unresponsive, the hardware watchdog automatically rebooted the device and normal operation resumed without manual intervention.

The main implemented results were:

- thermal frame-difference motion detection using the FLIR Lepton 3.5;
- PAG7936 RGB event recording;
- simultaneous creation of separate RGB and thermal event recordings;
- a five-second circular RAM prebuffer at five frames per second;
- 25 retained pre-event frames per filled buffer;
- 640 x 400 RGB prebuffering with 1280 x 800 live RGB recording;
- scaling of buffered RGB frames for inclusion in the higher-resolution MJPEG recording;
- prebuffer catch-up to reduce transition gaps;
- motion-based recording termination after five seconds without detected movement;
- a one-minute maximum recording duration;
- MJPEG timing correction;
- AVI idx1 index generation;
- automatic video numbering;
- SD-card video and logging quotas;
- rotating persistent logs;
- Finnish local timestamps;
- Lepton FFC-aware motion detection;
- proactive memory-management mechanisms;
- hardware watchdog recovery;
- asynchronous HTTPS upload of recorded MJPEG files;
- temporary presigned S3 upload URLs without permanent AWS credentials on the embedded device;

- automatic Lambda processing triggered by S3 ObjectCreated events;
- extraction of PAG7936 JPEG frames from uploaded MJPEG recordings;
- wildlife target detection using Amazon Rekognition;
- storage of detected target frames in Amazon S3; and
- modular embedded and AWS backend architectures.

13 Discussion

The project demonstrated that implementing pre-event video recording on a memory-constrained embedded camera requires balancing image quality, recording duration, frame rate, and available RAM.

The clearest example was the RGB prebuffer resolution. Using 1280 x 800 frames would have provided better pre-event image quality, but the memory requirement exceeded the available MicroPython heap. Reducing the prebuffer to 640 x 400 allowed five seconds of RGB565 frames to be retained reliably. The system could then switch to 1280 x 800 for live recording and scale the earlier frames when writing the final video.

The thermal camera also introduced challenges that would not exist in a conventional RGB-only motion detector. Thermal frame differencing proved capable of detecting movement, but the Lepton's periodic FFC calibration had to be treated as part of the overall sensor behavior. Without FFC awareness, a valid internal camera calibration could be incorrectly classified as movement.

Another significant result was the importance of long-duration testing. Many problems did not appear during short functional tests. Memory pressure, garbage collection, watchdog behavior, camera buffering, and other interactions became visible only after extended operation.

The project architecture also changed considerably during development. Moving from a monolithic camera class to separate camera-specific classes and a central camera manager improved separation of responsibilities. The

recording state machine further clarified the transition between file preparation, buffered recording, live recording, and finalization.

The prototype reached functional embedded-to-cloud operation, but further validation remains useful. Network-loss behavior, closely timed PAG and Lepton uploads, outdoor environmental behavior, and target-recognition accuracy should be tested further before the system is considered suitable for long-term field deployment.

14 Conclusion

The project achieved its primary objective of developing a motion-triggered embedded camera system with RAM-based pre-event recording.

The OpenMV N6 was successfully used with both a PAG7936 RGB camera and a FLIR Lepton 3.5 thermal camera. Thermal frame differencing provides the event trigger, while circular RAM buffers preserve footage from before the event. The system creates separate RGB and thermal MJPEG recordings and manages their storage on a microSD card.

Several supporting mechanisms were necessary to make the system suitable for extended autonomous operation. These included memory-aware buffer design, adaptive thermal background processing, FFC handling, MJPEG timing and AVI index correction, persistent logging, storage quotas, automatic garbage collection, and hardware watchdog recovery.

The development process also demonstrated the importance of iterative testing in embedded systems. Problems originating from USB hardware, microSD cards, firmware behavior, memory limitations, thermal-camera calibration, and software architecture required different debugging approaches. Long-duration stress testing was particularly valuable because several reliability problems were not visible during short tests.

At the current stage, the embedded camera system and AWS backend are operational and have been validated through recording, upload, S3 event processing, and Rekognition tests. Further work should focus on field enclosure and power design, network-failure handling, environmental testing, and recognition accuracy under varied outdoor conditions.

15 References

The following OpenMV documentation, official OpenMV examples, and AWS documentation were used during the implementation and documentation of the project.

[1] OpenMV. "OpenMV MicroPython libraries." OpenMV MicroPython 1.28 documentation. Available: https://docs.openmv.io/v5.0.0/library/index.html (accessed 27 August 2026).

[2] OpenMV. "Quick start." OpenMV MicroPython 1.28 documentation. Available: https://docs.openmv.io/v5.0.0/openmvcam/tutorial/quickstart.html (accessed 27 August 2026).

[3] OpenMV. "Video Recording Examples." OpenMV GitHub repository. Available: https://github.com/openmv/openmv/tree/master/scripts/examples/01-Camera/01-Video-Recording (accessed 27 August 2026).

[4] OpenMV. "Frame Differencing Examples." OpenMV GitHub repository. Available: https://github.com/openmv/openmv/tree/master/scripts/examples/02-Image-Processing/03-Frame-Differencing (accessed 27 August 2026).

[5] OpenMV. "FLIR Lepton Examples." OpenMV GitHub repository. Available: https://github.com/openmv/openmv/tree/master/scripts/examples/01-Camera/05-Thermal-Cameras/01-FLIR-Lepton (accessed 27 August 2026).

[6] OpenMV. "WiFi Examples." OpenMV GitHub repository. Available: https://github.com/openmv/openmv/tree/master/scripts/examples/09-WiFi (accessed 27 August 2026).

[7] Amazon Web Services. "Uploading objects with presigned URLs." Amazon S3 User Guide. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/PresignedUrlUploadObject.html (accessed 27 August 2026).

[8] Amazon Web Services. "Process Amazon S3 event notifications with Lambda." AWS Lambda Developer Guide. Available: https://docs.aws.amazon.com/lambda/latest/dg/with-s3.html (accessed 27 August 2026).

[9] Amazon Web Services. "DetectLabels." Amazon Rekognition API Reference. Available: https://docs.aws.amazon.com/rekognition/latest/APIReference/API_DetectLabels.html (accessed 27 August 2026).

[10] OpenMV Forums. “FLIR Lepton 3.5 shutter.” OpenMV Forums, 19 July 2019. Available: https://forums.openmv.io/t/flir-lepton-3-5-shutter/1356 (accessed 27 August 2026).

[11] Wikipedia contributors. "Circular buffer." Wikipedia. Available: https://en.wikipedia.org/wiki/Circular_buffer (accessed 31 August 2026).

[12] OpenMV. "gc — control the garbage collector." OpenMV MicroPython 1.28 documentation. Available: https://docs.openmv.io/v5.0.0/library/gc.html (accessed 31 August 2026).

[13] OpenMV. "class WDT – watchdog timer." OpenMV MicroPython 1.28 documentation. Available: https://docs.openmv.io/v5.0.0/library/machine.WDT.html (accessed 31 August 2026).

[14] OpenMV. "asyncio — asynchronous I/O scheduler." OpenMV MicroPython 1.28 documentation. Available: https://docs.openmv.io/v5.0.0/library/asyncio.html (accessed 31 August 2026).

[15] OpenMV. "mjpeg.c." OpenMV GitHub repository. Available: https://github.com/openmv/openmv/blob/master/lib/imlib/mjpeg.c (accessed 31 August 2026).

16 Current Main Configuration

Parameter	Value
Prebuffer duration	5 seconds
Prebuffer rate	5 FPS
Frames per full prebuffer	25
PAG7936 prebuffer resolution	640 x 400
PAG7936 prebuffer memory	~12.21 MiB
PAG7936 live recording resolution	1280 x 800
Lepton resolution	160 x 120
Thermal represented range	20-40 °C
Motion-check interval	200 ms
No-motion recording timeout	5 seconds
Maximum recording duration	60 seconds
PAG7936 stabilization	2 seconds
Lepton stabilization	5 seconds
POST-FFC recovery	5 seconds
Video storage quota	85%
Log storage quota	10%
Storage safety margin	5%

Maximum individual log size	256 KiB
Hardware watchdog timeout	30 seconds
Watchdog feed interval	5 seconds
PAG7936 60 s video upload duration	~511 s (~8 min 31 s)
PAG7936 60 s video upload speed	~154 KiB/s
Lepton 60 s video upload duration	~7.17 s
Lepton 60 s video upload speed	~128 KiB/s